

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN
HOMEWORK 1: MINESWEEPER AI
MÔN HỌC: NHẬP MÔN TRÍ TUỆ NHÂN TẠO - CO3061

THIẾT KẾ TRÒ CHƠI MINESWEEPER

SINH VIÊN 1: Nguyễn Đình Bằng - 2210298

SINH VIÊN 2: [Họ và tên 2] - [MSSV 2]

SINH VIÊN 3: [Họ và tên 3] - [MSSV 3]

TP. HỒ CHÍ MINH, 2026

Mục lục

1	Giới thiệu	1
1.1	Mục tiêu & Yêu cầu	1
1.2	Luật chơi cơ bản	1
2	Kiến trúc & Thiết kế	1
2.1	Các thành phần MVC	2
2.2	Chiến lược "An toàn lượt đầu"(First-Click Safety)	2
3	Thuật toán áp dụng	2
3.1	Thuật toán Depth-First Search (DFS) cho cơ chế Flood Fill	2
3.1.1	Tại sao dùng Không đệ quy (Iterative) thay vì Đệ quy (Recursive)? . . .	2
3.1.2	Các bước của thuật toán DFS	3
3.2	Cài đặt AI Solver	3
3.2.1	Tầng 1: Propositional Logic (Suy luận theo Mệnh đề)	3
3.2.2	Tầng 2: DFS Backtracking (Thỏa mãn ràng buộc)	4
3.2.3	Tầng 3: Educated Guess (Dự đoán theo Xác suất)	4
4	Phân công nhiệm vụ	4

1 Giới thiệu

1.1 Mục tiêu & Yêu cầu

Minesweeper (Dò mìn) là một trò chơi giải đố logic kinh điển. Mục tiêu của trò chơi là mở tất cả các ô trên một bảng mà không chạm vào bất kỳ quả mìn nào được giấu kín. Người chơi nhận được các gợi ý bằng số cho biết có bao nhiêu quả mìn nằm trong 8 ô xung quanh một ô.

Các mục tiêu chính của bài tập lớn này bao gồm:

1. **Cài đặt trò chơi:** Phát triển một trò chơi Minesweeper hoàn chỉnh bằng Python và thư viện giao diện đồ họa (GUI) Tkinter, không sử dụng các thư viện game có sẵn như Pygame.
2. **Sử dụng thuật toán duyệt đồ thị:** Áp dụng thuật toán Tìm kiếm theo chiều sâu (Depth-First Search - DFS) để mở rộng nhanh chóng khu vực các ô trống an toàn khi người chơi click vào một ô không có mìn xung quanh (cơ chế Flood Fill).
3. **Tích hợp AI Solver:** Thiết kế và cài đặt một tác tử AI có khả năng tự động giải quyết trò chơi bằng cách suy luận vị trí của mìn thông qua logic toán học, thỏa mãn ràng buộc (constraint satisfaction) và xác suất.

1.2 Luật chơi cơ bản

- **Mở ô (Click trái):** Click chuột trái vào một ô để mở nó. Nếu đó là mìn, trò chơi kết thúc (Thua). Nếu đó là một con số, nó hiển thị tổng số mìn ở 8 ô xung quanh. Nếu ô đó trống (không có mìn xung quanh), trò chơi sẽ tự động mở rộng tất cả các ô an toàn lân cận.
- **Cắm cờ (Click phải):** Người chơi có thể click chuột phải để đặt cờ (Flag) lên ô mà họ nghi ngờ có mìn. Cờ giúp ngăn việc vô tình click trái vào mìn.
- **Mở nhanh - Chord Reveal (Double-Click):** Nếu một ô số đã mở và có đúng số lá cờ xung quanh bằng với giá trị của ô đó, double-click vào ô số sẽ tự động mở tất cả các ô chưa mở xung quanh nó.
- **An toàn lượt đầu (First-Click Safety):** Lượt click đầu tiên của trò chơi được đảm bảo an toàn 100% (không bao giờ trúng mìn) và thường sẽ mở ra một vùng lớn để bắt đầu suy luận.

2 Kiến trúc & Thiết kế

Trò chơi được xây dựng dựa trên mô hình **Model-View-Controller (MVC)** nhằm phân tách rạch ròi giữa logic xử lý bên trong, giao diện người dùng và bộ xử lý tương tác. Việc phân tách này giúp mã nguồn dễ bảo trì và mở rộng sau này.

2.1 Các thành phần MVC

- **Model** (MinesweeperModel): Đại diện cho trạng thái bên trong của bàn cờ. Thành phần này lưu trữ vị trí của các quả mìn, số lượng mìn xung quanh mỗi ô, và trạng thái tương tác của người chơi (chưa mở, đã mở, đã cấm cờ). Nó cũng chịu trách nhiệm thực thi các quy tắc cốt lõi (đếm số mìn lân cận và kiểm tra điều kiện thắng/thua).
- **View** (MinesweeperView): Quản lý Giao diện Người dùng (GUI) được cài đặt chủ yếu thông qua thư viện Tkinter. Thành phần này vẽ lưới các nút bấm tương tác, thanh trạng thái (chứa bộ đếm mìn, nút khởi động lại hình mặt cười, và đồng hồ đếm giờ), cùng các hộp thoại. View hoàn toàn không biết gì về logic bên trong của trò chơi.
- **Controller** (MinesweeperController): Đóng vai trò trung gian. Nó lắng nghe các sự kiện chuột (click trái, click phải, double-click) từ View, yêu cầu Model cập nhật trạng thái dựa trên các tương tác đó, và sau đó điều khiển View vẽ lại để phản ánh trạng thái mới (ví dụ: hiển thị một con số hoặc một lá cờ).

2.2 Chiến lược "An toàn lượt đầu" (First-Click Safety)

Để tránh trải nghiệm khó chịu khi người chơi click ngay vào quả mìn ở lượt đầu tiên, hệ thống áp dụng chiến lược sinh mìn độ trễ. Khi khởi tạo trò chơi mới, bàn cờ hoàn toàn trống. Đồng hồ chưa chạy và chưa có quả mìn nào được đặt. Khi người chơi thực hiện click chuột trái đầu tiên tại tọa độ (r, c) , Model ghi nhận đây là nước đi khởi đầu. Một vùng cực an toàn kích thước 3×3 được thiết lập xung quanh (r, c) . Sau đó, các quả mìn mới được rải ngẫu nhiên *chỉ ở các ô nằm ngoài* vùng an toàn này. Điều này đảm bảo lượt click đầu tiên luôn an toàn tuyệt đối và thường mở ra một khu vực lớn để bắt đầu suy luận.

3 Thuật toán áp dụng

3.1 Thuật toán Depth-First Search (DFS) cho cơ chế Flood Fill

Khi người chơi click vào một ô trống (ô có số 0, tức không có mìn lân cận), trò chơi sẽ tự động mở tất cả các ô trống liền kề và dừng lại tại các ô có chứa số (nằm cạnh mìn). Quá trình này yêu cầu việc duyệt một lưới 2D, có thể được mô hình hóa toán học như một đồ thị vô hướng.

Để thực hiện điều này, chúng tôi áp dụng thuật toán **Tìm kiếm theo chiều sâu không đệ quy (Iterative Depth-First Search - DFS)** kết hợp với một cấu trúc dữ liệu Ngăn xếp (Stack - LIFO).

3.1.1 Tại sao dùng Không đệ quy (Iterative) thay vì Đệ quy (Recursive)?

Ngôn ngữ Python ép buộc một giới hạn đệ quy khá khắt khe để tránh lỗi tràn bộ nhớ (mặc định khoảng 1000 khung lệnh). Trên các bảng có kích thước lớn, ví dụ như tùy chỉnh 30×30 (900

ô), một hàm DFS đệ quy sâu có thể làm ứng dụng bị đứng (RecursionError). Việc sử dụng cấu trúc Mảng (Array) như một Stack tự quản lý sẽ né tránh hoàn toàn vấn đề này vì nó bị giới hạn chỉ bởi RAM máy tính.

3.1.2 Các bước của thuật toán DFS

1. Khi có sự kiện click trái mở một ô trống (r, c) .
2. DFS khởi chạy bằng cách đẩy (r, c) vào stack, đồng thời khởi tạo một danh sách visited (các ô đã duyệt qua) và danh sách revealed_cells chứa kết quả.
3. Vòng lặp thuật toán tiếp tục chạy chừng nào stack chưa rỗng:
 - (a) Lấy phần tử (r, c) ở đỉnh stack ra để xét.
 - (b) Nếu (r, c) đã có trong visited, bỏ qua nó. Nếu chưa, thêm (r, c) vào tập visited.
 - (c) Nếu trạng thái của ô không phải là "chưa mở"(hidden), bỏ qua.
 - (d) Đổi trạng thái ô thành "đã mở"(revealed) và lưu lại dữ liệu ô đó.
 - (e) Nếu giá trị thực sự của ô là 0 (trống), tiến hành xét 8 ô hàng xóm lân cận. Nếu ô hàng xóm nằm trong bảng, chưa bị duyệt qua, và không phải là mìn, đẩy tọa độ của nó vào stack.
4. Kết thúc và trả về toàn bộ danh sách revealed_cells để View cập nhật giao diện tổng cộng 1 lần.

3.2 Cài đặt AI Solver

Để nâng cao chất lượng bài làm và đáp ứng yêu cầu bộ môn một cách sáng tạo, một hệ thống Trí tuệ Nhân tạo (MinesweeperAI) đã được tích hợp. Thay vì dùng một bot dò đoán ngẫu nhiên thông thường, AI này giải bằng một **Hệ thống suy luận 3 tầng**.

3.2.1 Tầng 1: Propositional Logic (Suy luận theo Mệnh đề)

Bot liên tục lập bản đồ các ràng buộc, áp dụng hai quy luật tuyệt đối chính xác cho bất kỳ ô số V nào đã mở, có F lá cờ bao quanh và giáp với H ô chưa mở:

- **Quy tắc 1 (Tất cả an toàn - All Safe):** Nếu $V = F$, nghĩa là đã tìm đủ mìn, suy ra H ô chưa ráp còn lại xung quanh phải tuyệt đối an toàn. Do đó, **mở** (reveal) chúng.
- **Quy tắc 2 (Tất cả là mìn - All Mines):** Nếu $(V - F) = H$, nghĩa là số mìn còn thiếu vừa đúng bằng số ô chưa ráp, thì tất cả ô trống đó đều là mìn. Do đó, **cắm cờ** (flag) chúng.

3.2.2 Tầng 2: DFS Backtracking (Thỏa mãn ràng buộc)

Khi bộ quy tắc cơ bản ở Tầng 1 gặp ngõ cụt, AI chuyển sang chế độ ràng buộc (Constraint Programming).

Mỗi ô số đã mở tạo ra một phương trình ràng buộc lên các ô trống giáp nó. AI gom nhóm các "ô biên" này thành các cụm (cluster) độc lập bằng cấu trúc dữ liệu tập hợp rời rạc **Union-Find** (Disjoint Set). Với mỗi nhóm có kích thước nhỏ (≤ 15), nó khởi chạy DFS Backtracking:

- Cây tìm kiếm lần lượt ép giá trị Đúng (Min) hoặc Sai (An toàn) cho mỗi ô.
- Sau mỗi lượt gán, nó sẽ kiểm tra tính hợp lệ (VD: Không thể gán 2 quả mìn cạnh một ô số '1'). Nếu vi phạm, nó sẽ loại bỏ (prune) nhanh nhánh đó.
- Nếu một ô cụ thể mang giá trị Đúng trong 100% các trường hợp hợp lệ, nó chắc chắn là mìn. Ngược lại, nếu chỉ chiếm 0%, nó hoàn toàn an toàn.

3.2.3 Tầng 3: Educated Guess (Dự đoán theo Xác suất)

Trong những màn chơi mà Minesweeper yêu cầu bắt buộc phải đoán (ví dụ các trường hợp 50/50), AI tính toán tỷ lệ xuất hiện mìn theo xác suất trên từng ô chưa mở một cách cục bộ dựa trên số lượng ràng buộc giao nhau. Sau đó sẽ đưa ra quyết định đánh đổi rủi ro chạm mìn thấp nhất.

4 Phân công nhiệm vụ

Bảng dưới đây mô tả chi tiết khối lượng công việc và mức độ đóng góp của từng thành viên trong nhóm đối với bài tập lớn (Homework 1 - Minesweeper AI).

MSSV	Họ và Tên	Chi tiết đóng góp
2210298	Nguyễn Đình Bằng	• Cài đặt cốt lõi trò chơi và thuật toán duyệt DFS (Iterative). • Xây dựng khung giao diện Tkinter và kiến trúc MVC. • Viết báo cáo phần Kiến trúc và Thuật toán DFS.
[MSSV 2]	[Họ và tên 2]	• Nghiên cứu và cài đặt AI Solver (Tầng 1 và 3). • Viết báo cáo phần AI Solver. • Cải thiện giao diện hiển thị UX/UI cho Minesweeper.
[MSSV 3]	[Họ và tên 3]	• Hoàn thiện thuật toán DFS Backtracking (Constraint Solving) cho AI Tầng 2. • Thiết kế và tổng hợp báo cáo bằng $\text{\LaTeX}$. • Thực hiện các unit test và đánh giá phần trăm tỉ lệ chiến thắng của bộ AI Solver.

Bảng 1: Bảng đánh giá mức độ đóng góp của từng thành viên